

Tech Challenge — Fase 3

Operação em Nuvem da Plataforma de Gestão de Oficina Mecânica

Integrantes: Rafael Xavier · Silas Furini

Vídeo de demonstração: https://youtu.be/Q7HPrM_qmXk

Repositórios:

Papel	Repositório
Aplicação principal	https://github.com/rafaelxvr/Tech-challenge-15SOAT
Infraestrutura Kubernetes	https://github.com/rafaelxvr/Tech-challenge-15SOAT-k8s-infra
Function Serverless	https://github.com/rafaelxvr/Tech-challenge-15SOAT-functions
Infraestrutura do banco	https://github.com/rafaelxvr/Tech-challenge-15SOAT-db-infra

Revisor: o usuário soat-architecture consta como colaborador aceito nos quatro repositórios, sem convites pendentes.

Sumário

1. [Resumo executivo](#)
2. [Problema, objetivos e escopo](#)
3. [Arquitetura da solução](#)
4. [Autenticação e API Gateway](#)
5. [Repositórios e CI/CD](#)
6. [Infraestrutura e escalabilidade](#)
7. [Observabilidade](#)
8. [Modelagem de dados](#)
9. [Decisões registradas](#)
10. [Entregáveis e links](#)
11. [Limitações e evoluções possíveis](#)
12. [Apêndices técnicos](#)

1. Resumo executivo

A Fase 2 entregou o sistema da oficina rodando em Kubernetes local, com Kind, Terraform e um pipeline de integração contínua. A Fase 3 move essa mesma aplicação para nuvem gerenciada na AWS e acrescenta o que faltava para operá-la: uma porta de entrada única com autorização explícita, autenticação de cliente sem senha, quatro repositórios com entrega contínua independente, e observabilidade suficiente para responder o que o sistema está fazendo.

O único endereço público é um API Gateway HTTP em us-east-1. Atrás dele, a aplicação Spring Boot roda em um cluster EKS por trás de um balanceador interno, e o banco PostgreSQL gerenciado não tem endereço público. A autenticação do cliente acontece em funções Lambda: uma emite um código de uso único para o e-mail cadastrado, outra troca esse código por um token assinado com chave assimétrica e escopo restrito às próprias ordens.

A autorização não está espalhada pelo código. É um contrato de rotas versionado, aplicado duas vezes — o gateway vincula cada rota ao autorizador, e o autorizador reaplica o mesmo contrato do lado servidor. Rota que ninguém concedeu é recusada.

O ambiente de homologação está no ar e foi verificado de ponta a ponta. Produção permanece desligada por padrão, atrás de variáveis de habilitação explícitas, e este documento não afirma o contrário.

1.1 Matriz de rastreabilidade

Requisito	Evidência	Situação
API Gateway com controle e roteamento	Seção 4; contrato phase3-v2	Atendido
Rotas sensíveis protegidas por autenticação via CPF	Seção 4.1	Atendido
Function Serverless que valida CPF, consulta o cliente e devolve JWT	Seção 4.2	Atendido
Quatro repositórios com CI/CD e deploy automático	Seção 5	Atendido
main protegida e merge apenas por Pull Request	Seção 5.2	Atendido
Deploy automático de homologação	Seção 5.3	Atendido
Deploy automático de produção	Seção 5.3 e Seção 11.1	Implementado e desligado por gate
Banco de dados gerenciado	Seção 6.2	Atendido
Cluster Kubernetes com escalabilidade	Seção 6.1	Atendido
Terraform para provisionamento	Seção 6.3	Atendido
Latência das APIs, CPU e memória, healthchecks e uptime	Seção 7.1 e 7.2	Atendido
Alertas para falhas no processamento de ordens	Seção 7.2	Atendido
Logs estruturados em JSON com correlação	Seção 7.3	Atendido
Dashboards de volume diário, duração por status e falhas de integração	Seção 7.1	Atendido
Diagrama de componentes e de sequência	Seção 3.3	Atendido
RFCs e ADRs	Seção 9	Atendido
Justificativa do banco e modelo ER	Seção 8	Atendido
README por repositório	Seção 10	Atendido
Vídeo de demonstração	Capa e Seção 10	Atendido
Revisor soat-architecture nos quatro repositórios	Capa e Seção 10	Atendido

2. Problema, objetivos e escopo

2.1 Problema

A oficina cresceu para várias unidades e a base de clientes aumentou. O sistema da fase anterior rodava em um cluster local, sem controle de acesso por cliente, sem visibilidade de operação e em um repositório único. Três consequências práticas: qualquer chamada que alcançasse a rede alcançava a API; uma falha só era percebida quando alguém reclamava; e uma mudança na infraestrutura obrigava a mexer no mesmo repositório da aplicação.

2.2 Objetivos

Colocar a aplicação em nuvem gerenciada, com uma superfície pública única e autorização declarada; permitir que o cliente consulte e decida sobre a própria ordem sem receber uma senha; separar os domínios de mudança em repositórios com entrega contínua independente; e instrumentar o sistema para que uma falha apareça antes da reclamação.

2.3 Escopo entregue

O ambiente de homologação está provisionado e em operação: gateway, cluster, banco gerenciado, quatro funções Lambda, monitoramento e alertas. Os quatro repositórios têm pipeline próprio, branch protegida e deploy automático a partir da branch de homologação.

Fora do escopo desta entrega: a ativação de produção, descrita na Seção 11.1, e o modelo de múltiplas unidades, que o enunciado cita na motivação mas não lista entre os requisitos obrigatórios.

3. Arquitetura da solução

3.1 Fronteiras

O API Gateway HTTP é o único componente com endereço público. O cluster EKS fica em sub-redes privadas, atrás de um Application Load Balancer interno, e o banco não aceita conexão de fora da VPC. As funções Lambda de autenticação são publicadas como rotas anônimas do gateway; todas as demais rotas passam pelo autorizador.

3.2 Componentes

Componente	Identidade
Gateway	API Gateway HTTP, us-east-1
Kubernetes	EKS oficina-phase3, versão 1.35, dois node groups gerenciados
Aplicação	oficina-app no namespace oficina-staging, atrás do ALB interno
Banco	RDS PostgreSQL 16.15, privado, schema na versão 8 do Flyway
Serverless	Lambdas challenge, verification, authorizer e notification
Observabilidade	New Relic, conta 8521907, aplicação oficina-api-staging

3.3 Diagramas

Os diagramas são Mermaid versionados, renderizados pelo GitHub na própria página:

- Componentes, com nuvem, APIs, banco e monitoramento: docs/phase-3/architecture/components.md
- Sequência de autenticação: docs/phase-3/architecture/authentication-sequence.md
- Sequência de abertura e entrega da ordem de serviço: docs/phase-3/architecture/order-opening-sequence.md
- Modelo relacional: docs/phase-3/architecture/data-model.md

4. Autenticação e API Gateway

4.1 Duas identidades, um gateway

O cliente não tem senha. Ele se identifica pelo CPF, recebe um código de uso único no e-mail cadastrado e troca esse código por um token. O staff continua entrando por e-mail e senha. Os dois tokens são diferentes em assinatura, em tempo de vida e em poder:

	Cliente	Staff
Assinatura	RS256, chave assimétrica	HS256
Validade	15 minutos, sem refresh	24 horas
Escopos	orders:read:self, orders:decide:self	papel do operador
Alcance	apenas as próprias ordens	operação da oficina

4.2 A função de autenticação

POST /api/auth/cpf/desafios valida o CPF, consulta o cliente na base e confirma que está ativo. Um cliente inexistente ou inativo não recebe código. A função gera um código de seis dígitos, guarda apenas o **hash** dele com prazo de cinco minutos e envia o código por e-mail. A resposta devolve o identificador do desafio e o prazo, nunca o código.

POST /api/auth/cpf/verificar confere o código contra o hash e devolve o JWT. O token carrega o identificador do cliente, o tipo de principal e os dois escopos, e nenhum papel administrativo.

4.3 Autorização em contrato

As permissões vivem em `contracts/phase3-v2/routes.json`: para cada rota, a decisão e quem pode chamá-la. O padrão é negar. Uma rota que não aparece com concessão explícita é recusada, e a rota de atualização de status por e-mail foi deliberadamente removida da exposição.

O contrato é aplicado nas duas pontas. O gateway vincula cada rota protegida ao autorizador Lambda, e o autorizador carrega o mesmo arquivo e reaplica a decisão. Credencial ausente ou inválida resulta em 401; credencial válida sem concessão resulta em 403. A diferença importa: ela separa "não sei quem é você" de "sei quem é você e você não pode".

5. Repositórios e CI/CD

5.1 Separação por domínio de mudança

Repositório	Responsável por	Deploy
Aplicação	Spring Boot, migrações, imagem, manifests de workload	imagem no ECR e rollout no EKS
Kubernetes	rede, EKS, add-ons, ALB interno, casca do gateway, monitoramento	terraform apply por ambiente
Functions	Lambdas, autorizador, rotas públicas de CPF, IAM das funções	terraform apply e código
Banco	RDS, regras de rede, backup, referências de credencial	terraform apply por ambiente

Cada repositório é dono de um estado Terraform distinto. Nenhum recurso é gerenciado por dois estados, e a rede compartilhada pertence ao repositório de plataforma, que exporta os identificadores consumidos pelos demais.

5.2 Proteção de branch

main e develop estão protegidas nos quatro repositórios, com push direto bloqueado, exclusão bloqueada, merge apenas por Pull Request e as regras valendo também para administradores. A recusa é verificável: um push direto para develop retorna GH006: Protected branch update failed – Changes must be made through a pull request.

5.3 Deploy automático

Push em develop dispara o workflow de rollout de homologação, que testa o commit, constrói a imagem, publica no ECR com uma tag imutável no formato `staging-<sha>-<timestamp>`, atualiza o Deployment, espera a nova revisão ficar saudável e verifica o endpoint público. Falha em qualquer etapa reverte o Deployment para a revisão anterior.

O Deployment registra a origem do que está rodando, na anotação `oficina.io/released-commit`, o que permite ligar um pod a um commit e a um run do pipeline.

Produção roda apenas em main, condicionada à variável `APP_PRODUCTION_DEPLOYMENT_ENABLED` e ao ambiente protegido `production`. O gate está desligado.

A autenticação dos pipelines na AWS usa OIDC com credenciais de curta duração. A role de deploy confia no repositório exato, identificado por owner e repositório imutáveis, e não pelo nome. Nenhuma chave de acesso estática está guardada no GitHub.

6. Infraestrutura e escalabilidade

6.1 Cluster

O EKS oficina-phase3 roda a versão 1.35 com dois node groups gerenciados. A aplicação tem um HorizontalPodAutoscaler configurado entre uma e duas réplicas, com alvo de 60% de CPU, alimentado pelo Metrics Server. As sondas de liveness e readiness usam grupos distintos do Actuator: a readiness inclui a verificação do banco, a liveness não, para que uma indisponibilidade momentânea do banco não derrube o pod.

6.2 Banco gerenciado

RDS PostgreSQL 16.15, em sub-redes privadas, sem endereço público, com o schema evoluído por Flyway até a versão 8. A aplicação conecta com verify-full e um bundle de CA montado no pod. As credenciais de runtime, de migração e de administração são distintas, e a credencial de migração não é concedida aos pods da aplicação.

6.3 Terraform

Todo o provisionamento é declarado em Terraform, com estados separados por ambiente e por domínio. Os módulos de monitoramento também são Terraform: os dashboards, as condições de alerta e o monitor sintético são recursos versionados, não telas montadas à mão.

7. Observabilidade

7.1 Dashboards

Quatro dashboards publicados por Terraform:

Dashboard	Conteúdo
Platform	latência p95 da API, capacidade do Kubernetes em CPU e memória, heartbeat de telemetria e logs correlacionados
Orders	volume diário de ordens e idade por status
Business	duração média em diagnóstico, execução e finalização
Delivery	outbox bloqueado, falhas de entrega das funções e falhas de integração

Os agregados de negócio vêm de eventos publicados pela aplicação a cada 60 segundos, calculados a partir do histórico de transições da ordem de serviço, que grava cada mudança com o instante em que ocorreu.

7.2 Alertas

Quatorze condições em uma policy por ambiente, cobrindo latência e taxa de erro da API, CPU e memória de container e de nó, pods pendentes ou indisponíveis, saúde do gateway por monitor sintético, falhas técnicas no processamento de ordens, falhas de integração, outbox bloqueado e envelhecido, e três guardas de telemetria.

Uma dessas guardas dispara por **ausência** de sinal: se os eventos pararem de chegar por três minutos, o alerta abre. Silêncio deixa de ser confundido com saúde.

7.3 Logs e correlação

Os logs da aplicação são JSON estruturado. O encoder não imprime a mensagem livre nem o stack trace: os campos diagnósticos trafegam por chaves declaradas, o que impede que um dado de cliente ou uma resposta de provedor vaze para o log por descuido em uma chamada de log.

Cada requisição recebe um identificador de correlação, devolvido no cabeçalho X-Correlation-Id, e o gateway injeta o próprio identificador de requisição. Quando o chamador propaga o cabeçalho traceparent do W3C Trace Context, a linha de log carrega os três identificadores, o que liga a requisição ao trace distribuído. O widget de logs correlacionados exibe exatamente essas linhas.

8. Modelagem de dados

8.1 Justificativa do banco

A escolha por PostgreSQL gerenciado se sustenta em três pontos verificáveis no schema. O domínio usa tipos enumerados para o status da ordem e o tipo de documento, o que mantém a restrição no banco e não apenas na aplicação. O agregado da ordem de serviço depende de integridade referencial entre cliente, veículo, itens e histórico, com chaves estrangeiras e restrições de verificação. E o relatório operacional depende de agregação sobre janelas de tempo, com FILTER e funções de janela, que o PostgreSQL resolve sem estrutura auxiliar.

O serviço gerenciado acrescenta backup automatizado, atualização de versão controlada e isolamento de rede sem operação manual.

8.2 Ajustes do modelo nesta fase

O histórico de status ganhou o instante de ocorrência além do instante de gravação, para que a duração por status seja calculada sobre o tempo do negócio e não sobre o tempo de escrita. Foi acrescentada a versão de identidade do cliente, usada para invalidar tokens quando o documento ou o e-mail mudam. E entrou uma tabela de outbox transacional, que grava a notificação na mesma transação da mudança de estado e deixa a entrega para um publicador separado.

O diagrama entidade-relacionamento e a explicação dos relacionamentos estão em docs/phase-3/architecture/data-model.md.

9. Decisões registradas

RFCs, para decisões técnicas com alternativas em aberto:

RFC	Assunto
001	Perfil de nuvem
002	Modelo PostgreSQL
003	Autenticação por CPF
004	Notificações
005	Observabilidade

ADRs, para decisões arquiteturais já assumidas:

ADR	Assunto
001	Monolito modular
002	Escala por ambiente
003	Separação de confiança entre tokens
004	Entrega por outbox transacional
005	Histórico canônico de relatórios

10. Entregáveis e links

Entregável	Link
Vídeo de demonstração	https://youtu.be/Q7HPrM_gmXk
Repositório da aplicação	https://github.com/rafaelxvr/Tech-challenge-15SOAT
Repositório Kubernetes	https://github.com/rafaelxvr/Tech-challenge-15SOAT-k8s-infra
Repositório Functions	https://github.com/rafaelxvr/Tech-challenge-15SOAT-functions
Repositório do banco	https://github.com/rafaelxvr/Tech-challenge-15SOAT-db-infra
Índice da documentação da Fase 3	docs/phase-3/README.md
Diagrama de componentes	docs/phase-3/architecture/components.md
Sequência de autenticação	docs/phase-3/architecture/authentication-sequence.md
Sequência da ordem de serviço	docs/phase-3/architecture/order-opening-sequence.md
Modelo entidade-relacionamento	docs/phase-3/architecture/data-model.md
RFCs	docs/rfcs/
ADRs	docs/adrs/
Contratos de API, OpenAPI e Postman	docs/phase-3/api/contracts.md
Collection Postman da demonstração	postman/Oficina-Fase3-Staging.postman_collection.json
Matriz de requisito e evidência	docs/phase-3/evidence/requirements.md
Documento-fonte desta entrega	docs/entrega-fase-3.md

Cada um dos quatro repositórios tem README com o propósito, as tecnologias, os passos de execução e deploy, o diagrama daquele repositório e o link para a documentação da API.

O usuário soat-architecture consta como colaborador aceito nos quatro repositórios. Não há convites pendentes.

11. Limitações e evoluções possíveis

11.1 Limitações conhecidas

Produção não está ativa. O contrato de promoção existe, roda apenas em main e valida as entradas revisadas, mas depende de uma variável de habilitação e de um ambiente protegido. Os dois estão desligados. Esta entrega demonstra homologação.

O modelo de múltiplas unidades não foi implementado. O enunciado cita a expansão da oficina na motivação, mas não lista a segregação por unidade entre os requisitos obrigatórios. O domínio permanece de unidade única.

O código de uso único protege o canal, não a posse do CPF. Quem controla o e-mail cadastrado do cliente consegue autenticar. O enunciado pede validação de CPF e emissão de token; o código por e-mail é um reforço em cima disso, não uma prova de identidade.

A correlação completa depende do chamador. O identificador de correlação e o do gateway são preenchidos sempre, mas o traceparent só existe quando o cliente o propaga, conforme o W3C Trace Context.

11.2 Evoluções possíveis

Verificar em pipeline os contratos que hoje são acordados entre dois repositórios e escritos duas vezes, como o identificador da chave de assinatura e os cabeçalhos injetados pelo gateway. Nesta fase, duas divergências desse tipo só apareceram em investigação manual.

Ativar produção com a promoção controlada já descrita, e exercitar o rollback a partir de um release anterior.

Publicar métricas de negócio agregadas por unidade, quando o modelo de múltiplas unidades existir.

12. Apêndices técnicos

Apêndice A — Verificação rápida do ambiente

```
BASE=https://qcm8l43flb.execute-api.us-east-1.amazonaws.com
```

```
curl -s $BASE/health
curl -s -o /dev/null -w '%{http_code}\n' $BASE/api/clientes
curl -s -o /dev/null -w '%{http_code}\n' $BASE/api/clientes \
-H 'Authorization: Bearer invalido'
```

A primeira chamada responde {"status": "UP"}. As duas seguintes respondem 401: a rota é protegida e nenhuma das duas apresenta credencial válida.

Apêndice B — Fluxo de autenticação do cliente

```
curl -s -X POST $BASE/api/auth/cpf/desafios \
-H 'Content-Type: application/json' -d '{"cpf": "<cpf>"}'

curl -s -X POST $BASE/api/auth/cpf/verificar \
-H 'Content-Type: application/json' \
-d '{"desafioId": "<id>", "codigo": "<codigo recebido por e-mail>"}'
```

O desafio expira em cinco minutos. O token devolvido vale quinze minutos e não tem refresh.

Apêndice C — Rastreabilidade do que está implantado

```
kubectl get deployment oficina-app -n oficina-staging \
-o jsonpath='{.metadata.annotations.oficina.io/released-commit}'
```

Apêndice D — Checklist anterior ao envio

- ☒ Integrantes e data preenchidos.
- ☒ soat-architecture com acesso aceito nos quatro repositórios.
- ☒ Vídeo publicado e link inserido na capa e na Seção 10.
- ☒ Links dos quatro repositórios na capa e na Seção 10.
- ☒ Links das documentações na Seção 10.
- ☒ Limitações declaradas sem afirmar produção ativa.
- ☐ Gerar o PDF e conferir links, tabelas e quebras de página.
- ☐ Confirmar que nenhum segredo real aparece no repositório ou no PDF.